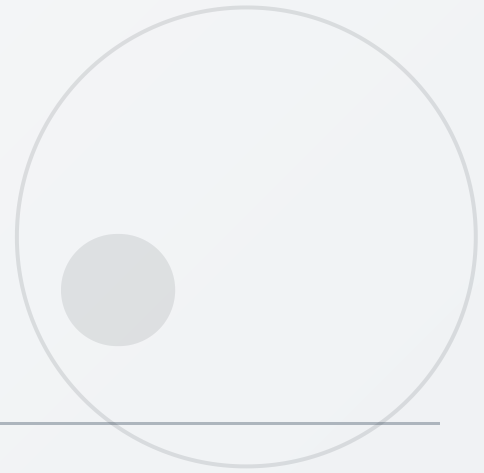




Especificacion Tecnica

Sistema de Comandas por Voz con Reconocimiento Offline VOSK
Interfaz 100% Dictada - Arquitectura Frugal PHP/MariaDB/Ubuntu

Documento: ET-Comandas-VOSK-v2.0

Fecha: Junio 2026

Revision: v2.0 (Modificacion de Alcance)

Plataforma: PHP / MariaDB / Ubuntu 22.04 LTS



Documento de especificacion tecnica de implementacion. Incluye modificaciones de alcance: motor TTS, gramatica restringida para cocineros, cola offline, y exclusion de procesamiento de pagos.

Indice de Contenidos

1. Resumen Ejecutivo Tecnico

2. Arquitectura del Sistema

2.1. Diagrama de Arquitectura

2.2. Flujo de Datos End-to-End

3. Stack Tecnologico

4. Modelo de Base de Datos

4.1. Esquema Completo (DDL)

4.2. Procedimientos Almacenados

5. API Backend (PHP)

5.1. Endpoints de Comandas

5.2. Endpoints de Cocina (Voz)

5.3. Endpoints de Cancelaciones

5.4. Endpoints de Cierre de Cuenta

6. Motor VOSK / WebAssembly

6.1. Pipeline de Audio

6.2. Gramatica Restringida

6.3. Correccion Fonetica (Levenshtein)

7. Sintesis de Voz (TTS)

8. Frontend PWA del Mesero

9. Frontend Pantalla Banner de Cocina

10. Infraestructura

10.1. Servidor Ubuntu 22.04

10.2. Dispositivos Android y Diademas

10.3. Red Local

11. Seguridad y Resiliencia

12. Plan de Implementacion

13. Referencias

1. Resumen Ejecutivo Tecnico

El presente documento establece las **especificaciones tecnicas de implementacion** para un sistema de comandas por reconocimiento de voz offline basado en el motor **VOSK**. El sistema opera integramente sobre infraestructura local con **PHP 8.x, MariaDB y Ubuntu Server 22.04 LTS**.

La arquitectura se fundamenta en tres pilares tecnicos diferenciadores: (1) el **reconocimiento de voz 100% offline** mediante WebAssembly en el navegador del dispositivo Android del mesero, (2) la **interaccion exclusiva por voz del cocinero** mediante diadema con gramatica ultra-restringida, y (3) la **sisntesis de voz (TTS)** para anuncios en altavoz de cocina y respuestas en diademas. El sistema no procesa metodos de pago; calcula totales y genera tickets impresos unicamente.

El procesamiento de audio se ejecuta en el edge (dispositivo del mesero/cocinero) mediante WebAssembly, distribuyendo la carga computacional y eliminando dependencias de servicios cloud. Las comandas se transmiten via HTTPS a un servidor local LAMP sobre Ubuntu 22.04, donde PHP-FPM gestiona la logica de negocio y MariaDB persiste los datos transaccionales.

Pilares Tecnologicos del Sistema

- **Edge computing con VOSK/WASM:** Transcripcion de voz local en el navegador del dispositivo Android.
- **Interfaz 100% voz para cocina:** Gramatica ultra-restringida para reconocimiento de comandos del cocinero.
- **TTS (Text-to-Speech):** Anuncios en altavoz de cocina y respuestas en diademas Bluetooth.
- **Cola offline con IndexedDB:** Persistencia local de comandas ante fallas de red Wi-Fi.
- **Arquitectura frugal LAMP:** Servidor unico con Ubuntu 22.04, Apache, PHP-FPM, MariaDB.

2. Arquitectura del Sistema

El sistema opera bajo una **arquitectura de tres capas** con procesamiento distribuido entre el edge (dispositivos móviles) y el servidor central.

2.1. Diagrama de Arquitectura

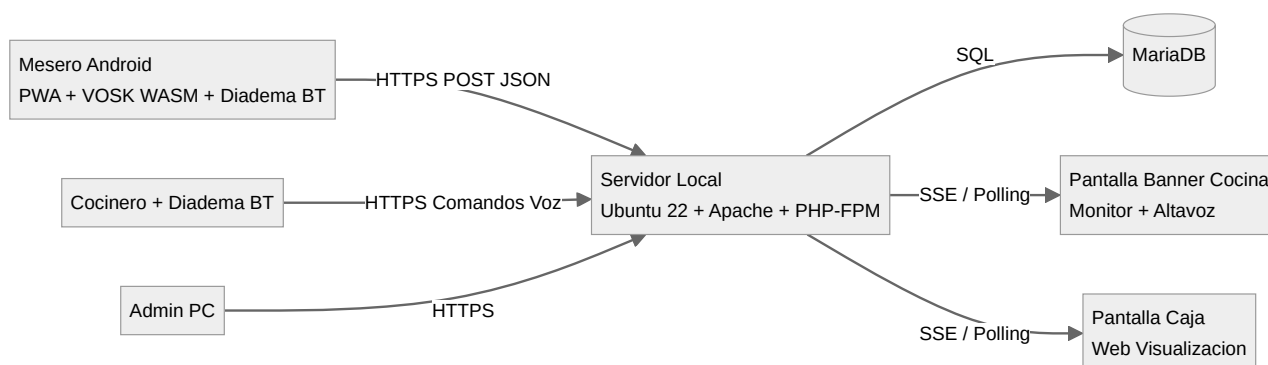


Figura 1. Arquitectura de alto nivel del sistema de comandas VOSK v2.0

Los componentes principales son:

- **Capa de Edge (Clientes):** Telefonos Android con PWA que ejecutan VOSK/WASM para reconocimiento de voz del mesero. Diademas Bluetooth para entrada de audio. Cocineros con diademas conectadas a estacion base para comandos de voz.
- **Capa de Aplicacion (Servidor):** Apache HTTP Server con PHP-FPM que expone API REST. Gestiona logica de comandas, cancelaciones, cierres de cuenta, y coordinacion TTS.
- **Capa de Datos:** MariaDB 10.6+ con esquema relacional para comandas, productos, usuarios y auditoria.
- **Capa de Visualizacion:** Pantalla banner en cocina (solo lectura) con altavoz para TTS. Pantalla de caja para visualizacion de totales.

2.2. Flujo de Datos End-to-End

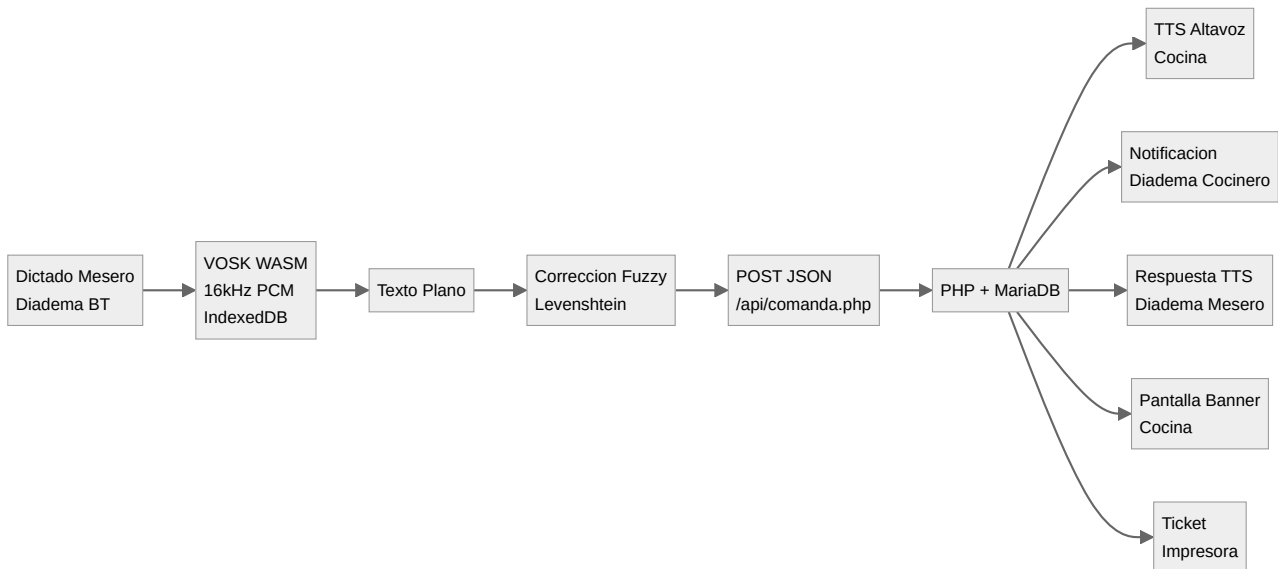


Figura 2. Flujo de datos completo incluyendo canales TTS

El flujo detallado es:

1. **Captura de audio:** El mesero dicta a su diadema Bluetooth. El AudioWorklet del navegador captura audio PCM a 16 kHz mono.
2. **Procesamiento local:** El Web Worker de VOSK (WASM) transcribe el audio a texto usando el modelo almacenado en IndexedDB.
3. **Correccion fonetica:** El algoritmo Levenshtein mapea palabras del dictado contra el catalogo de productos.
4. **POST al servidor:** La PWA envia un JSON via HTTPS POST al endpoint `/api/comanda.php`.
5. **Persistencia:** PHP valida y ejecuta el procedimiento almacenado `RegistrarComanda` dentro de una transaccion atomica en MariaDB.
6. **TTS Altavoz cocina:** El sistema genera audio TTS y lo reproduce por el altavoz conectado a la estacion de cocina: *"Nueva orden: mesa [N], [productos]."*
7. **Notificacion diadema cocinero:** Si hay comandos pendientes del cocinero (como solicitudes de cancelacion), se envian por su diadema.
8. **Respuesta diadema mesero:** El sistema confirma el envio por TTS en la diadema del mesero.
9. **Pantalla banner:** La nueva comanda aparece en la pantalla de cocina via SSE o polling.

10. **Comandos del cocinero:** El cocinero dicta comandos ("preparar siguiente", "listo mesa cinco") que son procesados por un segundo instancia VOSK con gramatica restringida.

3. Stack Tecnológico

Tabla 1. Stack Tecnológico Completo del Sistema

Capa		Tecnología	Version	Proposito
Sistema operativo		Ubuntu Server LTS	22.04	Infraestructura base del servidor local
Servidor web		Apache HTTP Server	2.4+	Host de aplicacion PHP, SSL, virtual hosts
Lenguaje backend		PHP	8.1 - 8.3	Logica de negocio, API REST, TTS
Procesador PHP		PHP-FPM	8.1 - 8.3	Gestion eficiente de procesos PHP concurrentes
Base de datos		MariaDB	10.6+	Persistencia relacional de comandas y usuarios
Frontend mesero		PWA (HTML5 + JS)	ES2020+	Interfaz progresiva, captura de voz, offline
Reconocimiento (mesero)	voz	VOSK (WebAssembly)	0.3.45+	Transcripcion offline de comandas a texto
Reconocimiento (cocinero)	voz	VOSK (WebAssembly)	0.3.45+	Transcripcion de comandos de control del cocinero
Modelo de lenguaje		vosk-model-small-es-0.42	0.42	Modelo pre-entrenado en espanol
Sintesis de voz (TTS)		Web Speech API (SpeechSynthesis)	W3C	Anuncios en altavoz y respuestas en diademas
Audio API		Web Audio API + AudioWorklet	W3C	Captura y procesamiento de audio de baja latencia
Almacenamiento local		IndexedDB API	W3C	Cache del modelo VOSK y cola offline
Sincronizacion		Fetch API + Background Sync	W3C	Envio de comandas y sincronizacion offline
Notificaciones real	tiempo	Server-Sent Events (SSE)	W3C	Push de comandas a pantalla banner y caja

Bluetooth audio	Web A2DP	Bluetooth API / W3C	Conexion de diademas al dispositivo Android
--------------------	-------------	---------------------------	--

3.1. Configuración de PHP-FPM

```
pm = ondemand
pm.max_children = 10
pm.process_idle_timeout = 10s
pm.max_requests = 500
```

3.2. Configuración de MariaDB

```
innodb_buffer_pool_size = 512M
innodb_log_file_size = 128M
innodb_flush_log_at_trx_commit = 2
query_cache_size = 0
query_cache_type = 0
```

4. Modelo de Base de Datos

4.1. Esquema Completo (DDL)

Tabla: usuarios

```
CREATE TABLE usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  usuario VARCHAR(50) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  rol ENUM('mesero', 'cocinero', 'cajero', 'administrador') DEFAULT 'mesero',
  diadema_id VARCHAR(50),
  activo TINYINT(1) DEFAULT 1,
  creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```


Tabla: mesas

```
CREATE TABLE mesas (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    numero INT UNIQUE NOT NULL,  
    capacidad INT DEFAULT 4,  
    activa TINYINT(1) DEFAULT 1,  
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

Tabla: categorias

```
CREATE TABLE categorias (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(50) NOT NULL,  
    orden INT DEFAULT 0,  
    activa TINYINT(1) DEFAULT 1  
) ENGINE=InnoDB;
```

Tabla: productos

```
CREATE TABLE productos (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    categoria_id INT,  
    nombre VARCHAR(100) NOT NULL,  
    descripcion TEXT,  
    precio DECIMAL(10,2) NOT NULL,  
    codigo VARCHAR(20),  
    disponible TINYINT(1) DEFAULT 1,  
    palabras_clave VARCHAR(255),  
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    actualizado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
    FOREIGN KEY (categoria_id) REFERENCES categorias(id)  
) ENGINE=InnoDB;
```

Tabla: comandas

```
CREATE TABLE comandas (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  mesa_id INT NOT NULL,  
  mesero_id INT NOT NULL,  
  cocinero_id INT,  
  texto_transcrito TEXT,  
  total DECIMAL(10,2) DEFAULT 0.00,  
  estado ENUM('pendiente', 'en_preparacion', 'listo', 'entregado', 'cancelado')  
  DEFAULT 'pendiente',  
  creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  actualizado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  FOREIGN KEY (mesa_id) REFERENCES mesas(id),  
  FOREIGN KEY (mesero_id) REFERENCES usuarios(id),  
  FOREIGN KEY (cocinero_id) REFERENCES usuarios(id)  
) ENGINE=InnoDB;
```

Tabla: detalle_comandas

```
CREATE TABLE detalle_comandas (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  comanda_id INT NOT NULL,  
  producto_id INT NOT NULL,  
  cantidad INT NOT NULL DEFAULT 1,  
  precio_unitario DECIMAL(10,2) NOT NULL,  
  subtotal DECIMAL(10,2) NOT NULL,  
  estado ENUM('activo', 'cancelado') DEFAULT 'activo',  
  cancelado_por_mesero_id INT,  
  cancelado_por_cocinero_id INT,  
  cancelado_en TIMESTAMP NULL,  
  notas VARCHAR(255),  
  FOREIGN KEY (comanda_id) REFERENCES comandas(id) ON DELETE CASCADE,  
  FOREIGN KEY (producto_id) REFERENCES productos(id),  
  FOREIGN KEY (cancelado_por_mesero_id) REFERENCES usuarios(id),  
  FOREIGN KEY (cancelado_por_cocinero_id) REFERENCES usuarios(id)  
) ENGINE=InnoDB;
```

Tabla: tickets

```
CREATE TABLE tickets (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    mesa_id INT NOT NULL,  
    mesero_id INT NOT NULL,  
    total DECIMAL(10,2) NOT NULL,  
    num_productos INT NOT NULL DEFAULT 0,  
    impreso_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (mesa_id) REFERENCES mesas(id),  
    FOREIGN KEY (mesero_id) REFERENCES usuarios(id)  
) ENGINE=InnoDB;
```

Tabla: cancelaciones_pendientes

```
CREATE TABLE cancelaciones_pendientes (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    detalle_comanda_id INT NOT NULL,  
    mesero_id INT NOT NULL,  
    cocinero_id INT NOT NULL,  
    estado ENUM('pendiente', 'aprobada', 'rechazada', 'expirada') DEFAULT  
'pendiente',  
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    respondido_en TIMESTAMP NULL,  
    FOREIGN KEY (detalle_comanda_id) REFERENCES detalle_comandas(id),  
    FOREIGN KEY (mesero_id) REFERENCES usuarios(id),  
    FOREIGN KEY (cocinero_id) REFERENCES usuarios(id)  
) ENGINE=InnoDB;
```

4.2. Procedimientos Almacenados

RegistrarComanda

```

DELIMITER //
CREATE PROCEDURE RegistrarComanda(
    IN p_mesa_id INT,
    IN p_mesero_id INT,
    IN p_texto_transcrito TEXT,
    IN p_json_productos JSON
)
BEGIN
    DECLARE v_comanda_id INT;
    DECLARE v_total DECIMAL(10,2) DEFAULT 0.00;
    DECLARE v_producto_id INT;
    DECLARE v_cantidad INT;
    DECLARE v_precio DECIMAL(10,2);
    DECLARE i INT DEFAULT 0;
    DECLARE n INT;
    START TRANSACTION;
    INSERT INTO comandas (mesa_id, mesero_id, texto_transcrito, total)
    VALUES (p_mesa_id, p_mesero_id, p_texto_transcrito, 0.00);
    SET v_comanda_id = LAST_INSERT_ID();
    SET n = JSON_LENGTH(p_json_productos);
    WHILE i < n DO
        SET v_producto_id = JSON_UNQUOTE(JSON_EXTRACT(p_json_productos,
        CONCAT('$[',i,'].producto_id')));
        SET v_cantidad = JSON_UNQUOTE(JSON_EXTRACT(p_json_productos,
        CONCAT('$[',i,'].cantidad')));
        SELECT precio INTO v_precio FROM productos WHERE id = v_producto_id AND
        disponible = 1;
        IF v_precio IS NOT NULL THEN
            INSERT INTO detalle_comandas (comanda_id, producto_id, cantidad,
            precio_unitario, subtotal)
            VALUES (v_comanda_id, v_producto_id, v_cantidad, v_precio, v_precio *
            v_cantidad);
            SET v_total = v_total + (v_precio * v_cantidad);
        END IF;
        SET i = i + 1;
    END WHILE;
    UPDATE comandas SET total = v_total WHERE id = v_comanda_id;
    COMMIT;
    SELECT v_comanda_id AS comanda_id, v_total AS total, 'success' AS estado;
END //
DELIMITER ;

```

CerrarCuentaMesa

```
DELIMITER //
CREATE PROCEDURE CerrarCuentaMesa(
    IN p_mesa_id INT,
    IN p_mesero_id INT
)
BEGIN
    DECLARE v_total DECIMAL(10,2);
    DECLARE v_num_productos INT;
    DECLARE v_ticket_id INT;
    START TRANSACTION;
    SELECT COALESCE(SUM(subtotal), 0), COUNT(*) INTO v_total, v_num_productos
    FROM detalle_comandas dc
    JOIN comandas c ON c.id = dc.comanda_id
    WHERE c.mesa_id = p_mesa_id AND dc.estado = 'activo';
    INSERT INTO tickets (mesa_id, mesero_id, total, num_productos)
    VALUES (p_mesa_id, p_mesero_id, v_total, v_num_productos);
    SET v_ticket_id = LAST_INSERT_ID();
    UPDATE comandas SET estado = 'entregado' WHERE mesa_id = p_mesa_id;
    COMMIT;
    SELECT v_ticket_id AS ticket_id, v_total AS total, v_num_productos AS
    num_productos;
END //
DELIMITER ;
```

5. API Backend (PHP)

5.1. Endpoints de Comandas

POST /api/comanda.php - Registrar nueva comanda

REQUEST:

POST /api/comanda.php

Content-Type: application/json

```
{
  "mesa_id": 5,
  "mesero_id": 3,
  "texto_transcrito": "mesa cinco dos tacos al pastor con todo y una coca",
  "productos": [
    {"producto_id": 12, "cantidad": 2},
    {"producto_id": 45, "cantidad": 1}
  ]
}
```

RESPONSE 200 OK:

```
{
  "status": "success",
  "comanda_id": 1284,
  "total": 145.00,
  "hora_registro": "2026-06-11 14:32:18",
  "tts_mensaje": "Nueva orden: mesa cinco, dos tacos al pastor, una coca.",
  "tts_mesero": "Comanda de mesa cinco enviada."
}
```

RESPONSE 400 Bad Request:

```
{
  "status": "error",
  "mensaje": "Parametros incompletos: se requiere mesa_id y productos"
}
```

GET /api/comandas/pendientes.php - Listar comandas pendientes

```
REQUEST:
GET /api/comandas/pendientes.php?limite=50

RESPONSE 200 OK:
{
  "status": "success",
  "total": 12,
  "comandas": [
    {
      "id": 1284,
      "numero_mesa": 5,
      "mesero": "Juan Perez",
      "hora_captura": "2026-06-11 14:32:18",
      "minutos_transcurridos": 4,
      "estado": "pendiente",
      "productos": ["2x Tacos al pastor", "1x Coca-Cola 600ml"]
    }
  ]
}
```

5.2. Endpoints de Cocina (Voz)

POST /api/cocina/comando.php - Procesar comando de voz del cocinero

```
REQUEST:
POST /api/cocina/comando.php
Content-Type: application/json

{
  "cocinero_id": 7,
  "texto_dictado": "preparar siguiente"
}

RESPONSE 200 OK (comando valido):
{
  "status": "success",
  "accion": "preparar_siguiente",
  "comanda_id": 1284,
  "mesa_id": 5,
  "productos": ["2x Tacos al pastor", "1x Coca-Cola"],
  "tts_respuesta": "Orden de mesa cinco: dos tacos al pastor, una coca. En preparacion.",
  "nuevo_estado": "en_preparacion"
}

RESPONSE 200 OK (sin comandas pendientes):
{
  "status": "success",
  "accion": "preparar_siguiente",
  "tts_respuesta": "No hay ordenes pendientes."
}

RESPONSE 200 OK (comando no reconocido):
{
  "status": "success",
  "accion": "desconocido",
  "tts_respuesta": "Comando no reconocido. Intente de nuevo."
}
```

El servidor mantiene un **parser de comandos de cocina** que analiza el texto dictado mediante expresiones regulares y lo mapea a acciones del sistema:


```
// Parser de comandos del cocinero (PHP)
$comandos = [
    '/^preparar siguiente$/i'      => 'preparar_siguiente',
    '/^preparar mesa (\d+)$/i'     => 'preparar_mesa',
    '/^listo mesa (\d+)$/i'        => 'listo_mesa',
    '/^recuperar mesa (\d+)$/i'    => 'recuperar_mesa',
    '/^repetir orden mesa (\d+)$/i'=> 'repetir_orden',
    '/^ordenes pendientes$/i'      => 'ordenes_pendientes',
    '/^si,? cancelar$/i'          => 'confirmar_cancelacion',
    '/^no,? no cancelar$/i'       => 'rechazar_cancelacion',
];
```

GET /api/cocina/estado.php - Estado general de la cocina

REQUEST: GET /api/cocina/estado.php

RESPONSE 200 OK:

```
{
  "status": "success",
  "pendientes": 5,
  "en_preparacion": 3,
  "listas": 2,
  "cancelaciones_pendientes": 1,
  "tts_resumen": "Tienes cinco ordenes pendientes, tres en preparacion y dos listas."
}
```

5.3. Endpoints de Cancelaciones

POST /api/cancelacion/solicitar.php - Solicitar cancelacion (mesero)

REQUEST:

POST /api/cancelacion/solicitar.php

Content-Type: application/json

```
{
  "mesero_id": 3,
  "mesa_id": 5,
  "producto_id": 12,
  "tipo": "parcial"
}
```

RESPONSE 200 OK:

```
{
  "status": "success",
  "solicitud_id": 89,
  "estado": "pendiente_cocinero",
  "tts_mesero": "Solicitud de cancelacion enviada a cocina. Espere confirmacion.",
  "tts_cocinero": "El mesero Juan solicita cancelar tacos al pastor de mesa cinco. Puede confirmar?"
}
```

RESPONSE 200 OK (producto ya listo):

```
{
  "status": "error",
  "codigo": "producto_listo",
  "tts_mesero": "El producto ya esta listo. No se puede cancelar."
}
```

POST /api/cancelacion/responder.php - Responder cancelacion (cocinero)

```
REQUEST:
POST /api/cancelacion/responder.php
Content-Type: application/json

{
  "cocinero_id": 7,
  "solicitud_id": 89,
  "respuesta": "aprobada"
}

RESPONSE 200 OK (aprobada):
{
  "status": "success",
  "accion": "aprobada",
  "tts_cocinero": "Cancelacion autorizada.",
  "tts_mesero": "Cancelacion autorizada por cocina.",
  "tts_altavoz": "Cancelacion autorizada: tacos al pastor de mesa cinco."
}

RESPONSE 200 OK (rechazada):
{
  "status": "success",
  "accion": "rechazada",
  "tts_cocinero": "Cancelacion rechazada. Continue preparando.",
  "tts_mesero": "Cancelacion rechazada por cocina."
}
```

5.4. Endpoints de Cierre de Cuenta

POST /api/cuenta/cerrar.php - Cerrar cuenta de mesa

```

REQUEST:
POST /api/cuenta/cerrar.php
Content-Type: application/json

{
  "mesero_id": 3,
  "mesa_id": 5
}

RESPONSE 200 OK:
{
  "status": "success",
  "ticket_id": 456,
  "total": 145.00,
  "num_productos": 3,
  "tts_mesero": "Cuenta de mesa cinco. Total ciento cuarenta y cinco pesos. Tres productos. Ticket enviado a impresion.",
  "ticket_contenido": {
    "mesa": 5,
    "mesero": "Juan Perez",
    "fecha": "11/06/2026",
    "hora": "15:42",
    "productos": [
      {"cantidad": 2, "nombre": "Tacos al pastor", "subtotal": 90.00},
      {"cantidad": 1, "nombre": "Coca-Cola 600ml", "subtotal": 25.00},
      {"cantidad": 1, "nombre": "Agua de horchata", "subtotal": 30.00}
    ],
    "total": 145.00
  }
}

```

6. Motor VOSK / WebAssembly

6.1. Pipeline de Audio

El pipeline de procesamiento de audio para el reconocimiento de voz consta de las siguientes etapas técnicas:

1. **Captura con getUserMedia:** Se solicita acceso al microfono de la diadema Bluetooth conectada al dispositivo Android.

2. **AudioWorkletNode:** El procesador de audio opera en un hilo independiente a 16 kHz, PCM mono, 16 bits.
3. **Buffer circular:** Las muestras PCM se acumulan en un buffer circular de 4096 muestras.
4. **Web Worker con VOSK WASM:** El buffer se transmite al Web Worker donde VOSK ejecuta el decodificador.
5. **Resultados:** VOSK devuelve resultados parciales (`partialResult`) durante el habla y un resultado final (`result`) al detectar pausa.

```
// Pipeline de audio para reconocimiento VOSK
const SAMPLE_RATE = 16000;
const constraints = {
  audio: {
    echoCancellation: true,
    noiseSuppression: true,
    channelCount: 1,
    sampleRate: SAMPLE_RATE
  }
};
const stream = await navigator.mediaDevices.getUserMedia(constraints);
const audioContext = new AudioContext({ sampleRate: SAMPLE_RATE });
const source = audioContext.createMediaStreamSource(stream);
await audioContext.audioWorklet.addModule('js/audio-processor.js');
const workletNode = new AudioWorkletNode(audioContext, 'pcm-processor');

workletNode.port.onmessage = (e) => {
  const pcmBuffer = e.data;
  if (recognizer.acceptWaveform(pcmBuffer)) {
    const result = recognizer.result();
    procesarTexto(result.text);
  } else {
    const partial = recognizer.partialResult();
    mostrarParcial(partial.partial);
  }
};
source.connect(workletNode);
```

6.2. Gramatica Restringida

Para maximizar la precision del reconocimiento, se implementan dos vocabularios diferenciados:

Gramatica del Mesero (Catalogo de Productos)

```
// Vocabulario dinamico cargado desde el servidor
const vocabularioMesero = [
  "uno", "dos", "tres", "cuatro", "cinco", "seis", "siete", "ocho", "nueve",
  "diez",
  "mesa",
  "tacos al pastor", "tacos de bistec", "tacos de chorizo",
  "coca", "coca-cola", "agua de horchata", "agua de jamaica",
  "con todo", "sin cebolla", "sin cilantro",
  "cancelar", "cerrar", "cuenta"
];
recognizer.setWords(vocabularioMesero);
```

Gramatica del Cocinero (Comandos de Control)

```
// Vocabulario ultra-restringido para comandos del cocinero
const vocabularioCocinero = [
  "preparar", "siguiente", "mesa",
  "listo", "recuperar",
  "repetir", "orden", "ordenes", "pendientes",
  "cancelar", "si", "no",
  "uno", "dos", "tres", "cuatro", "cinco",
  "seis", "siete", "ocho", "nueve", "diez",
  "once", "doce", "trece", "catorce", "quince"
];
// Nota: En VOSK, setWords() restringe el vocabulario completo del reconocedor
```

6.3. Correccion Fonetica (Levenshtein)

El algoritmo de distancia de Levenshtein corrige errores menores de transcripcion comparando cada palabra del dictado contra el catalogo de productos:

```

function distanciaLevenshtein(str1, str2) {
  const track = Array(str2.length + 1).fill(null)
    .map(() => Array(str1.length + 1).fill(null));
  for (let i = 0; i <= str1.length; i++) track[0][i] = i;
  for (let j = 0; j <= str2.length; j++) track[j][0] = j;
  for (let j = 1; j <= str2.length; j++) {
    for (let i = 1; i <= str1.length; i++) {
      const indicator = str1[i - 1] === str2[j - 1] ? 0 : 1;
      track[j][i] = Math.min(
        track[j][i - 1] + 1,      // insercion
        track[j - 1][i] + 1,      // eliminacion
        track[j - 1][i - 1] + indicator // sustitucion
      );
    }
  }
  return track[str2.length][str1.length];
}

function corregirProducto(palabraDictada, catalogo) {
  const UMBRAL = 3;
  let mejorMatch = palabraDictada;
  let distanciaMinima = UMBRAL;
  for (const producto of catalogo) {
    const dist = distanciaLevenshtein(palabraDictada, producto);
    if (dist < distanciaMinima) {
      distanciaMinima = dist;
      mejorMatch = producto;
    }
  }
  return mejorMatch;
}

```

7. Sintesis de Voz (TTS)

El sistema utiliza la **Web Speech API (SpeechSynthesis)** del navegador para generar anuncios de voz. Esta API es nativa de Chrome/Android y no requiere instalacion de librerias adicionales.

7.1. Motor TTS

```
function sintetizarVoz(texto, callback = null) {
  const utterance = new SpeechSynthesisUtterance(texto);
  utterance.lang = 'es-MX';
  utterance.rate = 1.1;
  utterance.pitch = 1.0;
  utterance.volume = 1.0;
  // Seleccionar voz en español si esta disponible
  const voces = speechSynthesis.getVoices();
  const vozEs = voces.find(v => v.lang.startsWith('es') &&
v.name.includes('Mexico'));
  if (vozEs) utterance.voice = vozEs;
  if (callback) utterance.onend = callback;
  speechSynthesis.speak(utterance);
}
```

7.2. Cola de Mensajes TTS

Para evitar solapamiento de mensajes, se implementa una cola FIFO de reproduccion:

```
class ColaTTS {
  constructor() {
    this.cola = [];
    this.reproduciendo = false;
  }

  encolar(texto) {
    this.cola.push(texto);
    if (!this.reproduciendo) this.procesar();
  }

  procesar() {
    if (this.cola.length === 0) {
      this.reproduciendo = false;
      return;
    }
    this.reproduciendo = true;
    const texto = this.cola.shift();
    sintetizarVoz(texto, () => this.procesar());
  }
}

const colaAltavozCocina = new ColaTTS();
const colaDiademaCocinero = new ColaTTS();
const colaDiademaMesero = new ColaTTS();
```


7.3. Canales de Salida de Audio

Tabla 2. Canales de Salida de Audio TTS

Canal	Dispositivo Fisico	Audiencia	Eventos
Altavoz Cocina	Altavoz conectado a estacion PC de cocina	Todos los cocineros	Nuevas comandas, cancelaciones autorizadas, cierres
Diadema Cocinero	Diadema Bluetooth del cocinero	Cocinero individual	Confirmaciones de comandos, solicitudes de cancelacion
Diadema Mesero	Diadema Bluetooth del mesero	Mesero individual	Confirmacion de envio, aprobacion/rechazo de cancelacion, cierre

8. Frontend PWA del Mesero

La aplicacion para el mesero es una **Progressive Web App (PWA)** que se accede mediante el navegador Chrome del telefono Android.

8.1. Estructura de Archivos

Tabla 3. Archivos de la PWA del Mesero

Archivo	Funcion
<code>index.html</code>	Shell principal, metadatos PWA
<code>manifest.json</code>	Configuracion de icono, nombre, tema
<code>sw.js</code>	Service Worker para cache offline y sync
<code>js/vosk.js</code>	Wrapper de inicializacion VOSK
<code>js/recognizer.js</code>	AudioWorklet, captura PCM, comunicacion VOSK
<code>js/tts.js</code>	Wrapper de SpeechSynthesis para diadema mesero
<code>js/app.js</code>	Logica principal: UI, envio, cola offline
<code>js/fuzzy.js</code>	Algoritmo Levenshtein para correccion
<code>css/app.css</code>	Estilos responsivos movil-first

8.2. HTTPS Autofirmado

```
# Generar certificado autofirmado
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/private/comandas-local.key \
  -out /etc/ssl/certs/comandas-local.crt \
  -subj "/C=MX/ST=Estado/L=Ciudad/O=Restaurante/CN=comandas.local"
```

Nota sobre Certificados Autofirmados

Los navegadores mostraran una advertencia al primer acceso. El usuario debe seleccionar "Avanzado" y "Proceder". Esto solo ocurre la primera vez por dispositivo.

9. Frontend Pantalla Banner de Cocina

La pantalla de cocina es una **interfaz web de solo lectura** optimizada para monitores horizontales de 19 a 24 pulgadas. No admite entrada táctil, clic ni teclado.

9.1. Técnica de Implementación

- **Actualización:** Server-Sent Events (SSE) o polling cada 5 segundos al endpoint `/api/comandas/pendientes.php`.
- **Renderizado:** DOM virtual ligero para actualización eficiente de tarjetas sin recargar la página completa.
- **Ordenamiento:** Las comandas se ordenan por `creado_en` ascendente (FIFO) en el query SQL.
- **Código de colores:** CSS dinámico basado en minutos transcurridos desde `creado_en`.
 - Verde (border-left: 6px solid #28a745): 0-10 minutos
 - Amarillo (border-left: 6px solid #ffc107): 10-20 minutos
 - Rojo (border-left: 6px solid #dc3545): más de 20 minutos
- **Tiempo transcurrido:** Actualización del contador vía `setInterval` en JavaScript del lado del cliente.

9.2. Layout CSS del Banner

```
/* Estructura del banner de cocina */
.banner-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
  padding: 16px;
}
.comanda-card {
  border-left: 6px solid #28a745;
  background: #fff;
  padding: 12px;
  border-radius: 4px;
  box-shadow: 0 1px 3px rgba(0,0,0,0.1);
}
.comanda-card.urgente { border-left-color: #ffc107; }
.comanda-card.critica { border-left-color: #dc3545; }
.mesa-numero { font-size: 48px; font-weight: bold; text-align: center; }
.tiempo-transcurrido { font-family: monospace; font-size: 24px; }
```

10. Infraestructura

10.1. Servidor Ubuntu 22.04

Tabla 4. Especificaciones del Servidor

Componente	Minimo	Recomendado
CPU	Intel Core i3 / AMD Ryzen 3 (4 nucleos)	Intel Core i5 / AMD Ryzen 5 (6 nucleos)
RAM	4 GB	8 GB
Almacenamiento	120 GB SSD	256 GB SSD NVMe
Red	Ethernet Gigabit + Wi-Fi 802.11ac	Ethernet Gigabit + Wi-Fi 6
USB	4 x USB 3.0	6 x USB 3.0

Instalacion del stack LAMP

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y apache2 php8.3 php8.3-fpm php8.3-mysql \
    php8.3-mbstring php8.3-curl mariadb-server mariadb-client \
    openssl ssl-cert cups-bsd
sudo a2enmod ssl rewrite proxy proxy_fcgi setenvif
sudo a2enconf php8.3-fpm
sudo systemctl restart apache2
```

Configuracion de Impresion (CUPS)

Para la impresion de tickets, se instala CUPS y se configura una impresora termica conectada por USB al servidor:

```
sudo apt install -y cups cups-bsd
sudo usermod -aG lpadmin www-data
# Agregar impresora termica via interfaz web de CUPS
# http://localhost:631/admin
# Guardar nombre de impresora en configuracion del sistema
```

10.2. Dispositivos Android y Diademas

Tabla 5. Requerimientos de Dispositivos

Especificacion	Minimo	Recomendado
SO Android	8.0 (Oreo, API 26)	11+ (API 30)
RAM	3 GB	4 GB+
Almacenamiento libre	200 MB	500 MB
Procesador	64-bit, Octa-core 1.8 GHz	Snapdragon 6-series
Navegador	Chrome 90+	Chrome 120+
Bluetooth	4.2 (para diadema)	5.0+

10.3. Red Local

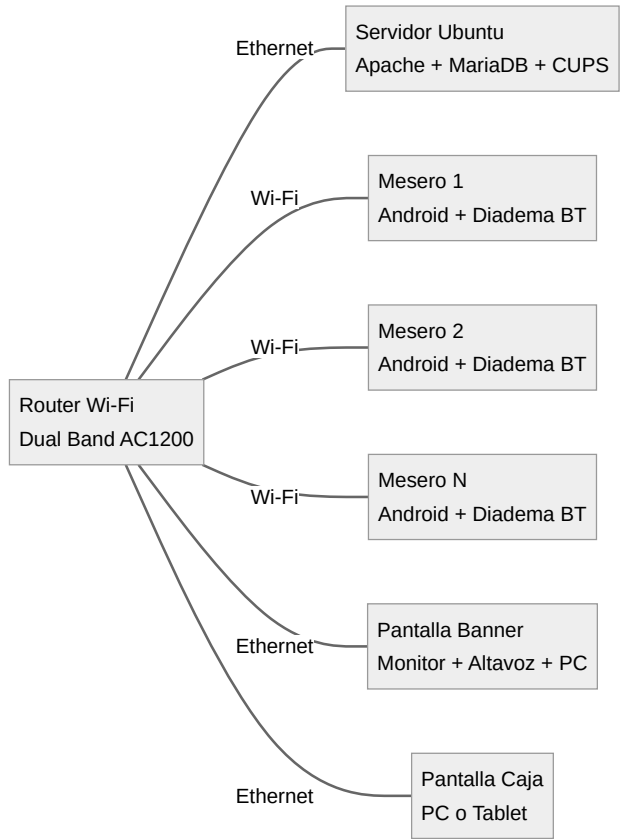


Figura 3. Topología de red local del restaurante

Tabla 6. Requisitos de Red

Parametro	Especificacion
Router	Dual band (2.4 GHz y 5 GHz), minimo AC1200
Cobertura Wi-Fi	100% area de comedor y cocina (repetidores si es necesario)
Subred	192.168.x.x/24, servidor con IP estatica
DNS local	comandas.local apuntando a IP del servidor
Puertos	80 (HTTP), 443 (HTTPS) abiertos
Ancho de banda local	Minimo 20 Mbps entre dispositivos y servidor

11. Seguridad y Resiliencia

- **Validacion de entradas:** Sanitizacion de datos en todos los endpoints para prevenir SQL injection y XSS.
- **Backup automatico:** Script cron cada hora: `mysqldump comandas_db > /backup/comandas_$(date +%Y%m%d_%H).sql`

11.2. Resiliencia Operativa

- **Reconocimiento 100% offline:** VOSK no requiere internet. La transcripcion ocurre localmente.
- **Cola offline:** Comandas se almacenan en IndexedDB ante perdida de Wi-Fi y se sincronizan al restaurar.
- **Independencia de cloud:** Sin dependencia de AWS, Google Cloud, Azure, ni APIs externas.
- **UPS recomendado:** 600VA+ para proteccion ante cortes electricos.
- **Cache persistente:** El modelo VOSK permanece en IndexedDB tras reinicios del navegador.

11.3. Privacidad

- **Audio nunca sale del dispositivo:** Las grabaciones se procesan localmente y no se transmiten ni almacenan.
- **Datos almacenados minimos:** Solo texto transcrito, hora y productos. Sin biometricos ni datos sensibles de clientes.
- **Control local:** Todos los datos residen fisicamente en el servidor del restaurante.

12. Plan de Implementacion

Tabla 7. Cronograma de Implementacion por Fases

Fase	Duracion	Actividades	Entregable
Fase 1: Preparacion	Semana 1	Hardware servidor; Ubuntu 22.04; red local; SSL; LAMP stack; CUPS	Servidor listo
Fase 2: Backend	Semanas 2-3	Base de datos MariaDB; API PHP; procedimientos almacenados; autenticacion	API funcional
Fase 3: PWA Mesero	Semanas 3-4	PWA; integracion VOSK WASM; correccion Levenshtein; cola offline; TTS diadema	PWA funcional

Fase 4: Cocina Voz	Semanas 4-5	VOSK secundario para cocinero; gramatica restringida; parser de comandos; TTS altavoz y diadema	Interfaz de cocina por voz
Fase 5: Pantallas	Semanas 5-6	Pantalla banner cocina; pantalla caja; impresion de tickets via CUPS	KDS + caja operativos
Fase 6: Catalogo	Semana 6	Carga de productos; palabras clave para VOSK; asignacion de diademas	Catalogo completo
Fase 7: Pruebas	Semana 7	Pruebas integrales; ajuste de gramatica; optimizacion TTS; validacion en produccion	Sistema validado
Fase 8: Capacitacion	Semana 8	Entrenamiento de meseros en dictado; capacitacion de cocineros en comandos de voz	Personal capacitado
Fase 9: Go-Live	Semana 9	Operacion en paralelo (opcional); monitoreo; soporte intensivo	Sistema en produccion

13. Referencias

1. Alpha Cephei. (2024). *VOSK Speech Recognition Toolkit*. Documentacion oficial. <https://alphacephei.com/vosk/>
2. Alpha Cephei. (2024). *VOSK Models Repository*. <https://alphacephei.com/vosk/models>
3. Alphacephei. (2023). *vosk-api*. Repositorio GitHub. <https://github.com/alphacep/vosk-api>
4. W3C. (2021). *Web Audio API*. Especificacion W3C. <https://www.w3.org/TR/webaudio/>
5. W3C. (2021). *Web Speech API*. Especificacion W3C. <https://wicg.github.io/speech-api/>
6. W3C. (2015). *Indexed Database API 2.0*. Especificacion W3C. <https://www.w3.org/TR/IndexedDB/>
7. Mozilla Developer Network. (2024). *Progressive Web Apps (PWAs)*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps
8. Apache Software Foundation. (2024). *Apache HTTP Server Documentation*. Version 2.4. <https://httpd.apache.org/docs/2.4/>
9. The PHP Group. (2024). *PHP Manual*. Version 8.3. <https://www.php.net/docs.php>
10. MariaDB Foundation. (2024). *MariaDB Server Documentation*. Version 10.6. <https://mariadb.com/kb/en/documentation/>